Implementing Graph Data Models in Neo4j 4.0

Course Guide

Table of Contents

Implementing Graph Data Models in Neo4j 4.0 . . . 1
- About this course . . . 1
- Lesson Overview . . . 1
- Resources . . . 1

Implementing Your First Model . . . 3
- About this module . . . 3
- Domain used for this training . . . 3
  - Data used for this training . . . 3
- Application question for our model . . . 4
- Sample data for our model . . . 5
- Working with the sample data . . . 5
- Exercise 1: Getting started with the airport graph data model . . . 6
- Check your understanding . . . 6
  - Question 1 . . . 6
  - Question 2 . . . 6
  - Question 3 . . . 7
- Summary . . . 7

Importing Data into the Graph . . . 8
- About this module . . . 8
- Options for importing data into the graph . . . 8
- Prepare for the import . . . 8
- Review: Steps for loading CSV data with Cypher . . . 9
- Review: Using `LOAD CSV` . . . 9
- Example: Inspecting data from the CSV file on network . . . 9
- Example: Inspecting data from the CSV file in import folder . . . 10
- Example: Creating nodes and relationships . . . 11
- Exercise 2: Loading airport data . . . 11
- Check your understanding . . . 11
  - Question 1 . . . 11
  - Question 2 . . . 12
  - Question 3 . . . 12
- Summary . . . 12

Profiling Queries . . . 13
- About this module . . . 13
- Workflow for profiling queries . . . 13
- Example: Profiling a query . . . 13
  - Analyzing the query profile - 1 . . . 14
  - Analyzing the query profile - 2 . . . 15

Analyzing the query profile - 3 . . . 16
Exercise 3: Profiling queries . . . 16
Do we need to change the model? . . . 16
Check your understanding . . . 18
Question 1 . . . 19
Question 2 . . . 19
Question 3 . . . 19
Summary . . . 19
Refactoring Graphs . . . 20
About this module . . . 20
Evolving the model . . . 20
Refactoring steps . . . 20
Refactor details . . . 21
Unique FlightId . . . 21
Creating the constraint . . . 22
Refactoring the graph . . . 22
Index for query performance . . . 23
Creating the index . . . 23
Profiling the query . . . 23
Result of the profile . . . 24
Cleaning up the graph . . . 26
Exercise 4: Creating Flight nodes from CONNECTED_TO relationships . . . 26
Adding another domain question . . . 26
Implementing the query . . . 27
Profiling the query . . . 27
Performing another refactor . . . 29
Refactor details . . . 30
Refactor implementation . . . 31
Profile our first query . . . 31
Profile our original second query . . . 33
Profile our revised second query . . . 35
Exercise 5: Creating the AirportDay node from the Airport and Flight nodes . . . 37
More questions for the model? . . . 37
Another question for the model . . . 38
Refactoring for specific relationships . . . 40
APOC to the rescue! . . . 40
Creating specialized relationships with APOC . . . 41
Refactoring result . . . 41
Does the query improve? . . . 41
Exercise 6: Creating specific relationships . . . 43
Refactoring large graphs . . . 43

Batching the refactoring process . . . 43
Example code for a batch . . . 44
Using apoc.periodoc.commit . . . 44
Batching with APOC . . . 45
Result of the batch processing . . . 45
Exercise 7: Refactoring large graphs . . . 45
Check your understanding . . . 46
Question 1 . . . 46
Question 2 . . . 46
Question 3 . . . 46
Summary . . . 46
Implementing Graph Data Models: Summary . . . 48
Course Summary . . . 48
Resources . . . 48
Course feedback . . . 49

Implementing Graph Data Models in Neo4j 4.0

About this course

This course introduces you to how to develop the Cypher code for creating the nodes and relationships for a graph data model. In addition, you will gain experience in refactoring an existing graph as the model changes.

This course is intended for developers and architects. You should have basic Neo4j experience and Cypher programming skills.

In this course, you complete hands-on exercises to gain experience with developing graph data models for use with Neo4j.

To complete the hands-on exercises for this course, you need:

- Neo4j 4.0 Instance that is started with one of:
  - Neo4j Desktop
  - Neo4j Sandbox
  - Neo4j Aura
- A database that is empty.
- Neo4j Browser window connected to the started Neo4j Server instance.
- Internet access.

This Course is published by Neo4j per this License for Use.

Lesson Overview

Here are the lessons of this course:

1. Implementing Your First Model
2. Importing Data
3. Profiling Queries
4. Refactoring Graphs

Resources

We have set up a discussion area in our Neo4j Community Site, if you run into problems in the course and need assistance. You should register on the Community site where you can view other questions and answers for students taking our online training courses. The Neo4j Community Site is an excellent resource for answering many types of questions posed by other users of Neo4j.

Here are some resources you may use as you go through this course:

- Neo4j Cypher Manual
- Neo4j Developer Resources
- Neo4j Community Site

Implementing Your First Model

About this module

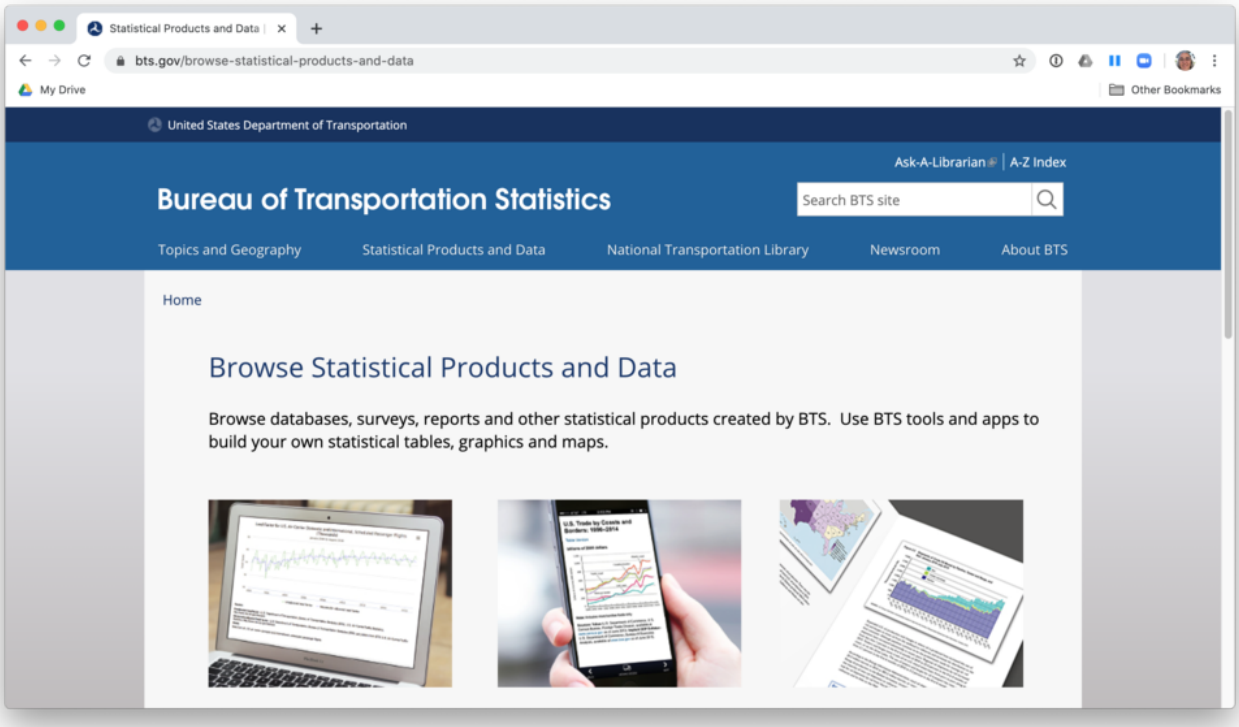
At the end of this module, you should be able to:

- Write Cypher code to implement a simple initial graph data model.
- Confirm that the starter data is in the graph.

Domain used for this training

For this training we will work with a familiar domain, airplane flights. We are working in a familiar domain so that we can focus on implementation details, and not be distracted by wider concepts.

Specifically, we will use a dataset from the US Bureau of Transportation.



Data used for this training

The airline data itself is quite rich. We will not make use of every part of it in the exercises, but you can check it out if you have spare time during the exercises.

Origin		
OriginAirportID	Origin Airport, Airport ID. An identification number assigned by US DOT to identify a unique airport. Use this field for airport analysis across a range of years because an airport can change its airport code and airport codes can be reused.	Analysis
OriginAirportSeqID	Origin Airport, Airport Sequence ID. An identification number assigned by US DOT to identify a unique airport at a given point of time. Airport attributes, such as airport name or coordinates, may change over time.	
OriginCityMarketID	Origin Airport, City Market ID. City Market ID is an identification number assigned by US DOT to identify a city market. Use this field to consolidate airports serving the same city market.	Analysis
Origin	Origin Airport	Analysis
OriginCityName	Origin Airport, City Name	
OriginState	Origin Airport, State Code	Analysis
OriginStateFips	Origin Airport, State Fips	Analysis
OriginStateName	Origin Airport, State Name	
OriginWac	Origin Airport, World Area Code	Analysis

Destination		
DestAirportID	Destination Airport, Airport ID. An identification number assigned by US DOT to identify a unique airport. Use this field for airport analysis across a range of years because an airport can change its airport code and airport codes can be reused.	Analysis
DestAirportSeqID	Destination Airport, Airport Sequence ID. An identification number assigned by US DOT to identify a unique airport at a given point of time. Airport attributes, such as airport name or coordinates, may change over time.	
DestCityMarketID	Destination Airport, City Market ID. City Market ID is an identification number assigned by US DOT to identify a city market. Use this field to consolidate airports serving the same city market.	Analysis
Dest	Destination Airport	Analysis
DestCityName	Destination Airport, City Name	
DestState	Destination Airport, State Code	Analysis
DestStateFips	Destination Airport, State Fips	Analysis
DestStateName	Destination Airport, State Name	
DestWac	Destination Airport, World Area Code	Analysis

Time Period		
Year	Year	
Quarter	Quarter (1-4)	Analysis
Month	Month	Analysis
DayofMonth	Day of Month	
DayOfWeek	Day of Week	Analysis
FlightDate	Flight Date (yyyymmdd)	

Application question for our model

As an air travel enthusiast,

I want to know how airports are connected,

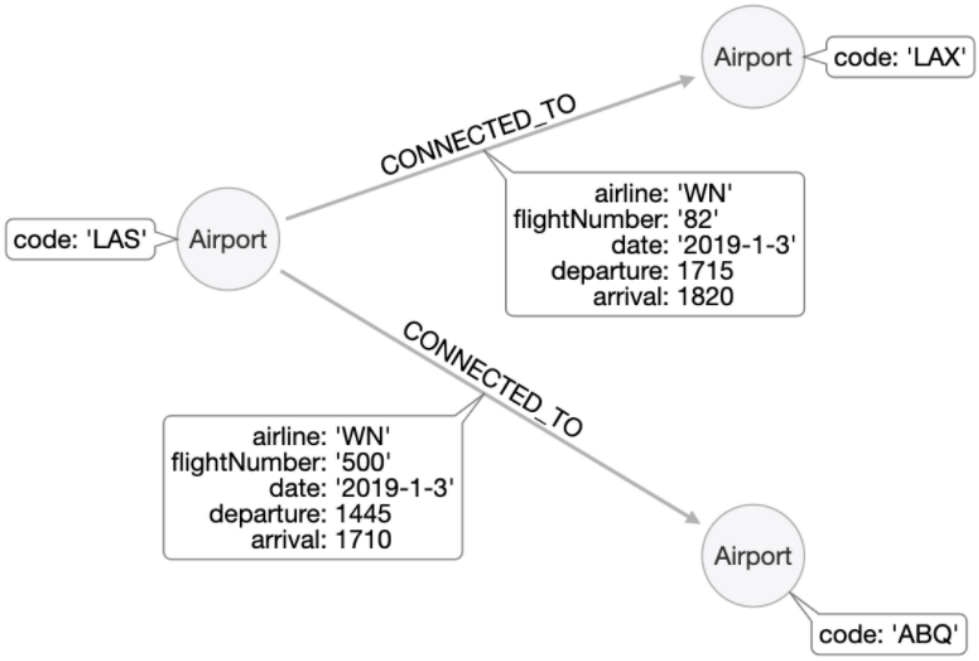
so I can find the busiest airports.

For this question:

- What are the entities?
- What are the connections between entities?
- What properties are needed?

Sample data for our model

Here is how we might model for this question with some sample data:



Working with the sample data

A best practice when creating a graph with Cypher is to use the `MERGE` statement. When creating nodes, you specify the properties in the `MERGE` statement to ensure that nodes with the same property values are not duplicated in the graph. For large graphs, the node properties used for the MERGE should have uniqueness constraints on them.

In the next exercise, Exercise 1, you will:

1. Create three *Airport* nodes and two *CONNECTED_TO* relationships per the initial data model by using the `MERGE` statement.
2. Query the graph to show the newly-created nodes and relationships.

Creating the nodes and relationships using the `MERGE` statement and hard-coding values is fine for small sample data, but once you start working with larger amounts of data, you will want to load the data (typically from CSV files).

Exercise 1: Getting started with the airport graph data model

Before you start this exercise you must:

1. Create a project in Neo4j Desktop, create a blank sandbox, or create a Neo4j Aura instance.
2. If using Neo4j Desktop, create a local 4.x database in the project and start it.
3. Open a Neo4j Browser window for the database.

In the query edit pane of Neo4j Browser, execute the browser command:

`:play 4.0-neo4j-modeling-exercises`

and follow the instructions for Exercise 1.

NOTE	This exercise has 3 steps. Estimated time to complete: 15 minutes.

Check your understanding

Question 1

What Cypher statement is a best practice for adding nodes and relationships to the graph?

Select the correct answer.

- ☐ `CREATE`
- ☐ `ADD`
- ☑ `MERGE`
- ☐ `INSERT`

Question 2

Given this code:

```
CREATE (:Person {name:"Joe"}
CREATE (:Person {name:"Jane"}
MERGE (:Person {name:"Bob"}
MERGE (:Person {name:"Joe"}
```

How many nodes are created in the graph?

Select the correct answer.

- ☐ 0
- ☐ 2
- ☑ 3

- [ ] 4

Question 3

Given this code:

```
MERGE (a:Airport {code:'LAS'})
MERGE (b:Airport {code:'LAX'})
MERGE (c:Airport {code:'ABQ'})
MERGE (a)-[:CONNECTED_TO {airline:'WN',flightNumber:'82',date:'2019-1-3'
,departure:'1715',arrival:'1820'}]->(b)
MERGE (a)-[:CONNECTED_TO {airline:'WN',flightNumber:'500',date:'2019-1-3'
,departure:'1445',arrival:'1710'}]->(c)
```

What is the implied entity for this domain?

Select the correct answer.

- [x] Airport
- [ ] code
- [ ] airline
- [ ] CONNECTED_TO

Summary

You should now be able to:

- Write Cypher code to implement a simple initial graph data model.
- Confirm that the starter data is in the graph.

Importing Data into the Graph

About this module

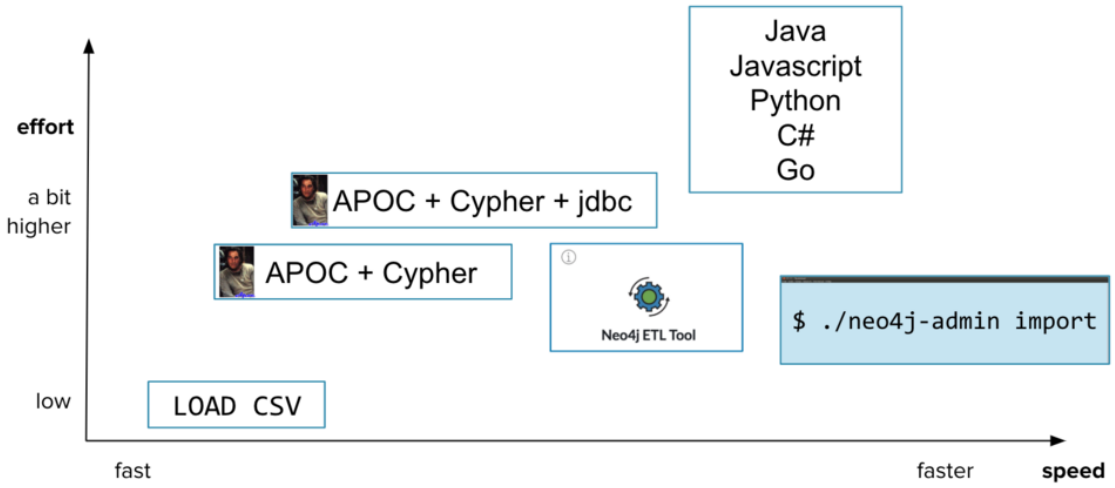
At the end of this module, you should be able to:

- Write Cypher code to import CSV data into the graph.
- Confirm that the data has been loaded.

Options for importing data into the graph

You have many options for importing data into Neo4j. Which option you choose depends on:

- How much data you have.
- What tools you are comfortable using.
- How much time you have to perform the import.



In this training, we will use Cypher to import data into the graph. Using Cypher enables us to control and customize how data is created and refactored as our model changes.

Prepare for the import

Before you import data into the graph, you should have an idea of the target graph data model you want to achieve. You should work with the data architects for your application so that everybody agrees upon:

- Names of entities (node labels).
- Names of relationships.
- Names of properties for nodes and relationships.
- Constraints to be defined.

- Indexes required.
- The most important queries.

Review: Steps for loading CSV data with Cypher

CSV import is commonly used to import data into a graph. If you want to import data from CSV, you will need to first develop a model that describes how data from your CSV maps to data in your graph.

Assuming that you have an agreed-upon data model, here are the basic steps you should follow for importing using Cypher and CSV files:

1. Determine how the CSV file will be structured.
2. Determine if normalized or denormalized data.
3. Ensure IDs to be used in the data are unique.
4. Ensure data in CSV files is "clean".
5. Execute Cypher code to inspect the data.
6. Determine if data needs to be transformed.
7. If required, ensure constraints are created in the graph.
8. Determine the size of the data to be loaded.
9. Execute Cypher code to load the data.
10. Add indexes to the graph.

NOTE	These steps are covered in the course *Introduction to Neo4j 4.0: Using LOAD CSV for Import*.

Review: Using `LOAD CSV`

Here is the simplified syntax for using `LOAD CSV`:

```
LOAD CSV     // load csv data
WITH HEADERS // optionally use first header row as keys in "row" map
FROM "url"   // file:/// file relative to $NEO4J_HOME/import or http://
AS row       // return each row of the CSV as list of strings or map
// ... rest of the Cypher statement ...
```

NOTE	You can use `LOAD CSV` for CSV files that contain fewer than 100k lines.

Example: Inspecting data from the CSV file on network

```
1 LOAD CSV WITH HEADERS FROM 'https://r.neo4j.com/flights_2019_1k' AS row
2 RETURN row
3 LIMIT 5
```



Started streaming 5 records after 1 ms and completed after 954 ms.

Example: Inspecting data from the CSV file in import folder

```
neo4j$ LOAD CSV WITH HEADERS FROM 'file:///flights_2019_1k.csv' AS row RETURN row LIMIT 5
```



Started streaming 5 records after 1 ms and completed after 28 ms.

Example: Creating nodes and relationships

You use `LOAD CSV` to read the data from the CSV file as a row to create nodes and relationships, for example:

```
LOAD CSV WITH HEADERS FROM 'https://r.neo4j.com/flights_2019_1k' AS row
MERGE (origin:Airport {code: row.Origin})
MERGE (destination:Airport {code: row.Dest})
MERGE (origin)-[connection:CONNECTED_TO {
  airline: row.UniqueCarrier,
  flightNumber: row.FlightNum,
  date: toInteger(row.Year) + '-' + toInteger(row.Month) + '-' +
toInteger(row.DayofMonth)}]->(destination)
ON CREATE SET connection.departure = toInteger(row.CRSDepTime),
connection.arrival = toInteger(row.CRSArrTime)
```

As each *row* is read from the file, *Airport* nodes are created with *code* property values of *row.Origin* and *row.Dest*. From the *row* values, we create the connection between the two nodes based upon the *row.uniqueCarrier* value for setting the *airline* property, *row.flightNumber* for the *FlightNum* property, and *row.Year* + *row.Month* + *row.DayOfMonth* for the *date* property. We use `MERGE` to ensure that duplicate nodes and relationships are not created with the same property values. If the connection is being created, we provide additional properties, *departure* and *arrival*.

For **large** datasets, you should ensure that uniqueness constraints (indexes) are created on the Airport code property before you load the data. This will dramatically improve the performance of the load as it will use the index during the `MERGE`. This dataset is small so load performance is not an issue at this point.

Exercise 2: Loading airport data

Your first import of airline data will use a CSV file with 1K lines so you will use the standard `LOAD CSV` statement. This CSV file has already been cleaned up and is in a normalized format.

In the query edit pane of Neo4j Browser, execute the browser command:

`:play 4.0-neo4j-modeling-exercises`

and follow the instructions for Exercise 2.

NOTE	This exercise has 9 steps. Estimated time to complete: 30 minutes.

Check your understanding

Question 1

What Cypher statement do you use to import data from a CSV file?

Select the correct answer.

- ☐ `LOAD DATA`
- ☐ `IMPORT DATA`
- ☑ `LOAD CSV`
- ☐ `IMPORT CSV`

Question 2

Up to how many lines can you import data using `LOAD CSV`?

Select the correct answer.

- ☐ 1K
- ☐ 10K
- ☑ 100K
- ☐ 1M

Question 3

When you import data using `LOAD CSV`, where can the CSV data come from?

Select the correct answers.

- ☑ File that has been placed in the **import** folder relative to the database instance.
- ☐ File that has been placed in the Neo4j Desktop project.
- ☑ File at a network location accessible via http/https.
- ☐ A JDBC connection that is open.

Summary

You should now be able to:

- Write Cypher code to import CSV data with Cypher.
- Confirm that the data has been loaded.

Profiling Queries

About this module

At the end of this module, you should be able to:

- Profile queries against the graph.
- Determine if a query can be improved.

Workflow for profiling queries

Understanding the performance of the queries for your use cases is an important part of implementing a graph data model. Here is the workflow you should use:

1. Load data into the graph.
2. Create queries that answer the application questions.
3. Execute the queries against the data to see if they retrieve the correct information.
4. PROFILE the query execution.
5. Identify problems and weaknesses in the query execution.
    a. Can the query be rewritten to perform better?
    b. Do we need to refactor the graph?
6. If necessary, modify the graph data model and refactor the graph.
7. PROFILE the same type of query against the refactored graph.

NOTE	The new query will be different due to the change in the graph data model.

Example: Profiling a query

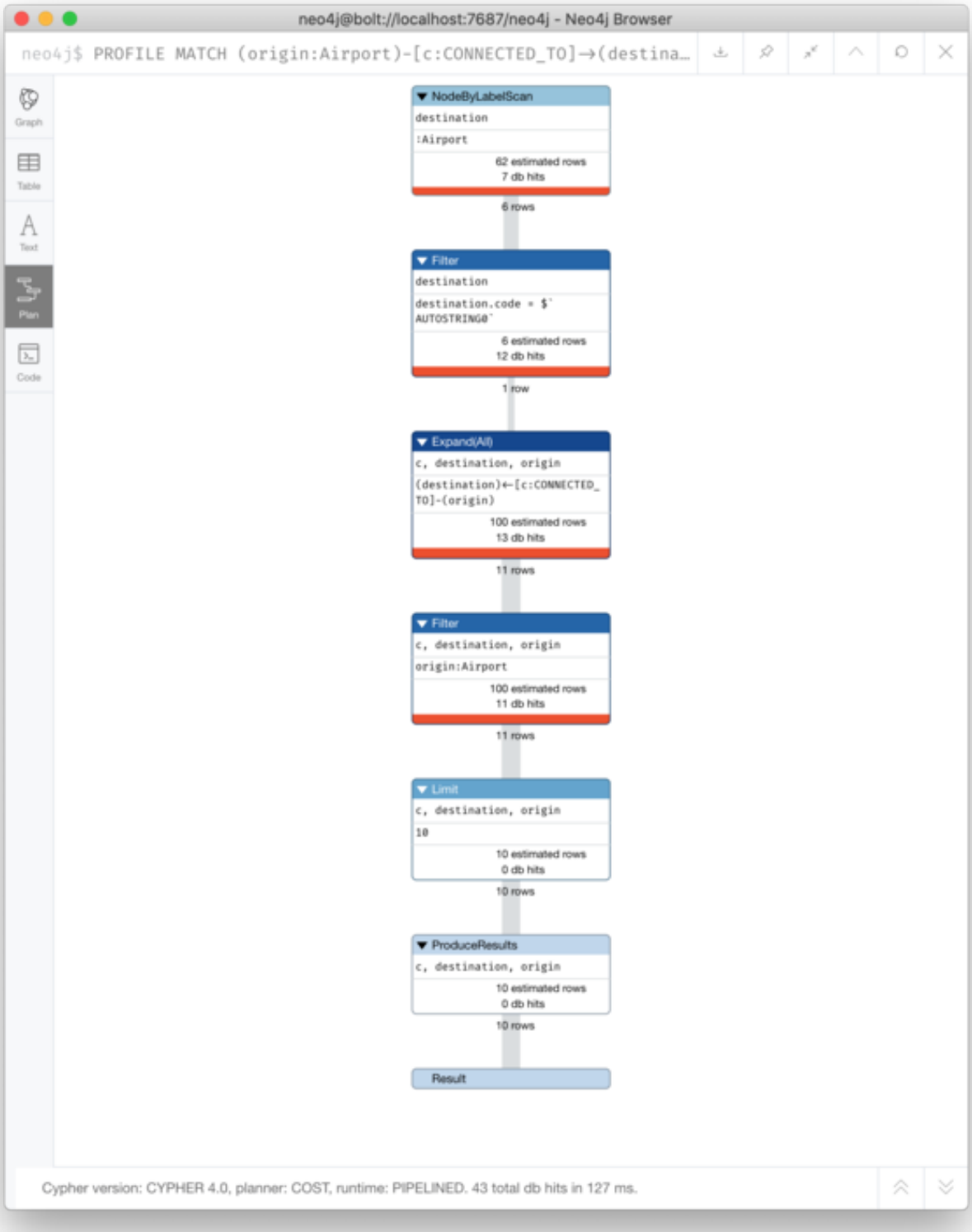
Here is the code to profile a query that retrieves all connections that have a destination of *LAS*:

```
PROFILE
MATCH (origin:Airport)-[c:CONNECTED_TO]->(destination:Airport)
WHERE destination.code = 'LAS'
RETURN origin, destination, c LIMIT 10
```

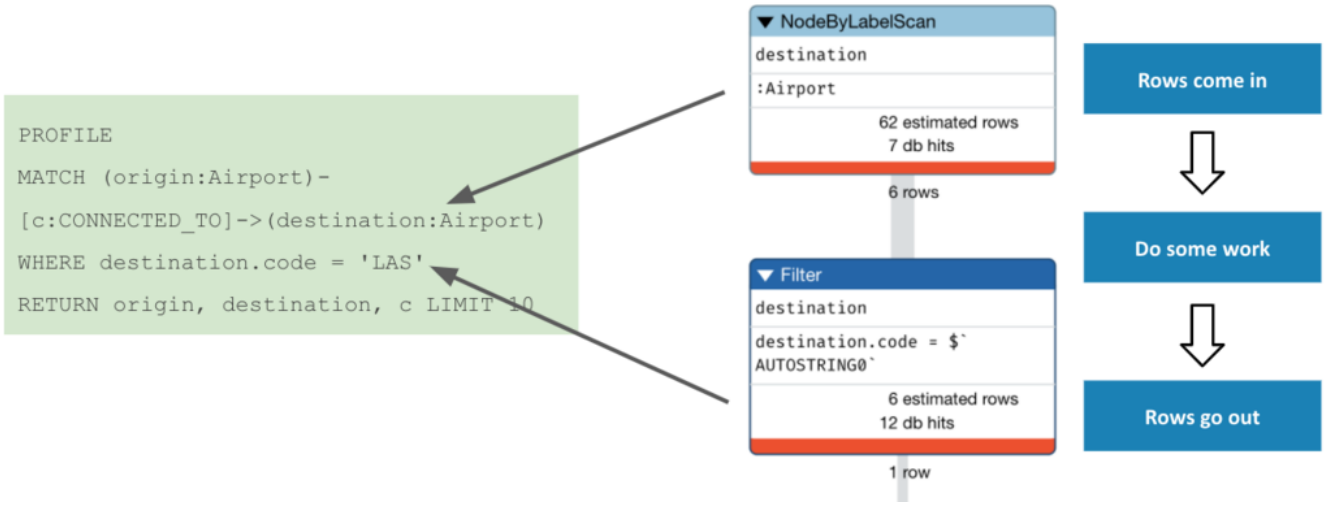
To profile a query, simply add `PROFILE` to the beginning of the Cypher statement.

This executes the query as normal, but gives a different output, which shows every step in the query execution plan, and how much each step cost.

With `PROFILE`, the main metric we consider is **db hits**. Execution time is important, but that is generally a result of administrative factors, like bandwidth, memory, and traffic volume. **db hits**, on the other hand, are entirely a function of the data model and query.



Analyzing the query profile - 1

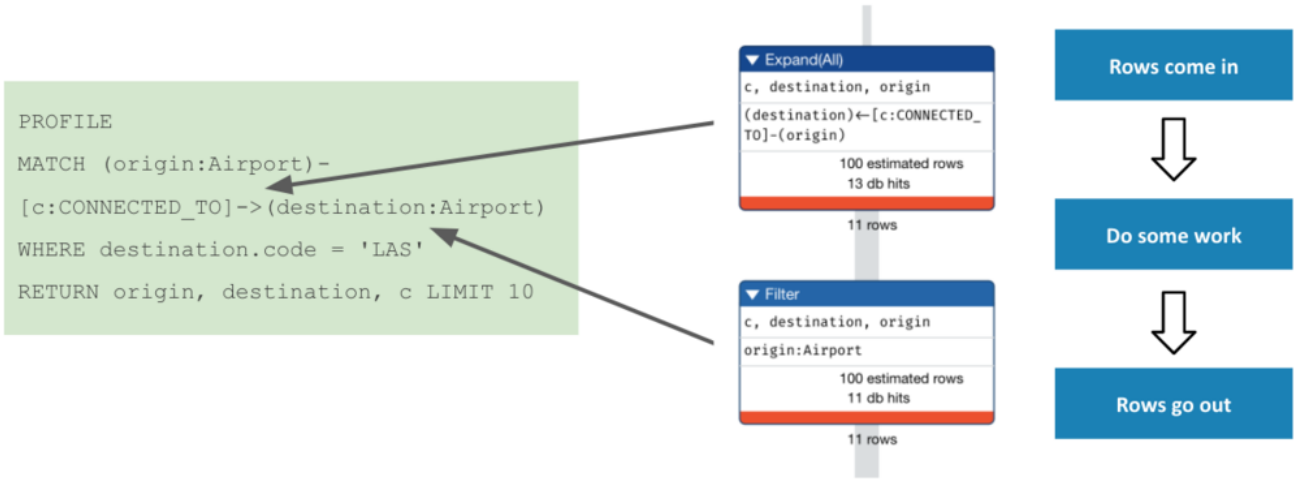


The first step in any query is locating the anchor. In this case, Neo4j will anchor on the destination node set, because that one has a property filter while origin does not.

According to the `PROFILE`, the anchor was located by first locating all *Airport* nodes under the Airport label, then scanning to find the desired code property value. This required 7 and 12 db hits respectively, and got us down to a single node—the perfect anchor set.

This label scan + property scan is **much** less efficient than an index lookup. We will observe that in a later exercise.

Analyzing the query profile - 2



With the anchor set identified, Neo4j then expanded along every incoming *CONNECTED_TO* relationship, finding 11 such paths with 13 and 11 db hits respectively.

Next, Neo4j checked which of those paths terminated at an *origin* node labeled *Airport*. Based on what we know about our model, it is no surprise that all 11 paths qualified. This is an example of providing an unnecessary filter. We could have dispensed with the *Airport* label on the *origin* set, and saved 11 db hits.

Analyzing the query profile - 3



With traversal finished, Neo4j then returned the results, filtered by the `LIMIT` clause. This operates entirely upon the objects already in memory, and so requires no db hits.

When all is said and done, you can see that total cost of this query was 43 db hits. We could have saved 11 by not filtering on *Airport* for the *origin* nodes. In addition, recall that identifying the anchor required 7+12 = 19 db hits. So our best opportunity for improving the performance of this query lies in finding a way to anchor more efficiently. PROFILE made that obvious, and we would have been unable to discover that by any other means.

Exercise 3: Profiling queries

In the query edit pane of Neo4j Browser, execute the browser command:

`:play 4.0-neo4j-modeling-exercises`

and follow the instructions for Exercise 3.

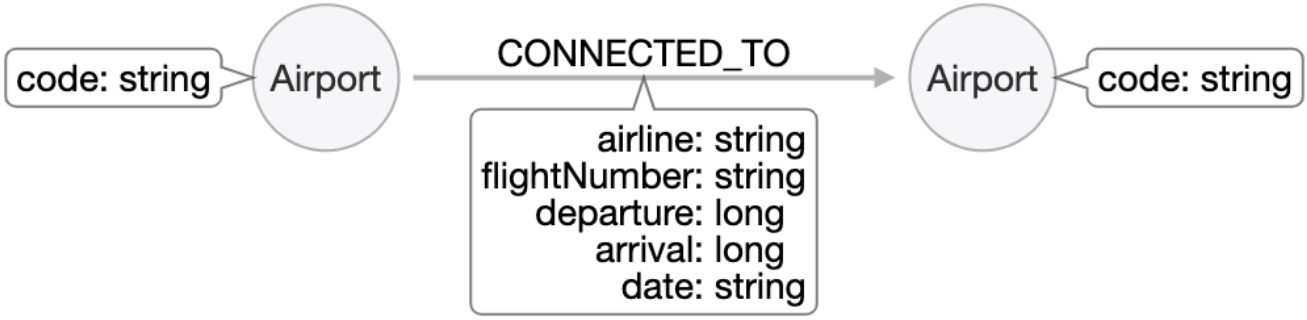
NOTE	This exercise has 2 steps. Estimated time to complete: 15 minutes.

Do we need to change the model?

In the previous exercise, we asked this question:

What are the airports and flight information for flight number 1016 for airline WN?

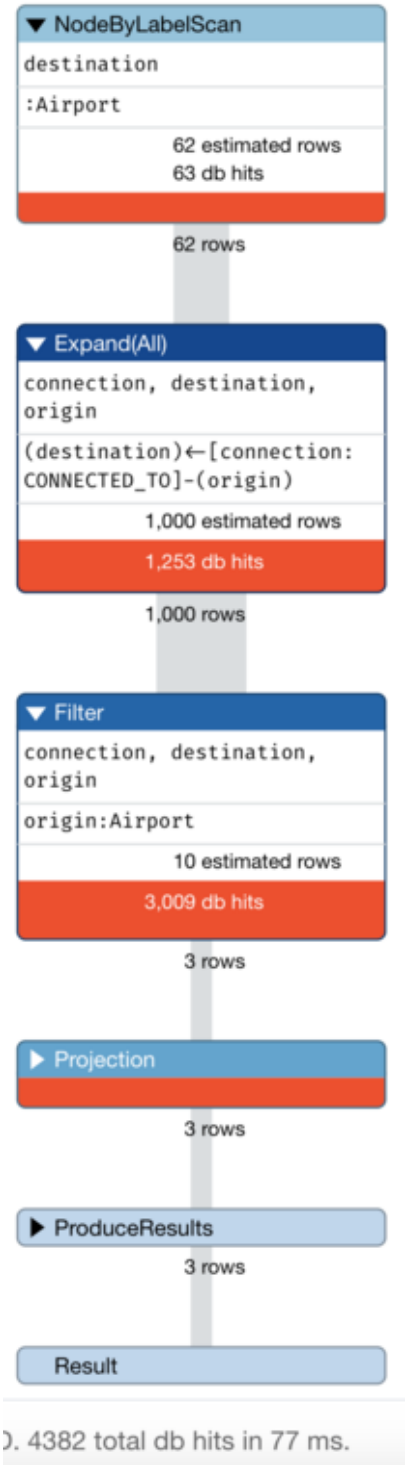
This is our current model:



Here is the query:

```
PROFILE
MATCH  (origin:Airport)-[connection:CONNECTED_TO]->(destination:Airport)
WHERE connection.airline = 'WN' AND connection.flightNumber = '1016'
RETURN origin.code, destination.code, connection.date,
connection.departure, connection.arrival
```

Here is the profile for this query:



This query required 4382 db hits.

Recall that our dataset only has 100 flights in it. In other words, in order to answer this question, we needed to hit the entire graph twice. The query executed reasonably quickly in terms of milliseconds, that is largely because the dataset is not large. Hence our insistence on using db hits as a measure—from that, we can tell that this query is grossly inefficient on this data model!

In the next lesson, we will implement a refactor to fix this.

Check your understanding

Question 1

What Cypher clause do you use to analyze the performance of a query?

Select the correct answer.

- ☐ `ANALYZE`
- ☐ `EXPLAIN`
- ☑ `PROFILE`
- ☐ `SHOW`

Question 2

When a query executes, the execution plan uses what data structure to pass data from one step of the plan to the next?

Select the correct answer.

- ☐ b-tree for the anchor node(s)
- ☐ neo4j-object structure containing nodes visited
- ☐ linked list of nodes visited
- ☑ row containing node and relationship values

Question 3

What are some observations from the profile of a query that would indicate that the graph data model needs improvement, the graph needs an index, or the query needs to be rewritten?

Select the correct answers.

- ☑ High number of db hits.
- ☑ Increasing number of db hits as the query executes.
- ☑ Seeing NodeByLabelScan as the first step of the query profile.
- ☑ No rows returned after the first step.

Summary

You should now be able to:

- Profile queries against the graph.
- Determine if a query can be improved.

Refactoring Graphs

About this module

At the end of this module, you should be able to:

- Create constraints that help with performance of node creation and queries.
- Determine if a query can be improved.
- Write Cypher code to refactor the data model in the graph.
- Create indexes that will help with query performance.
- Refactor a graph by creating intermediate nodes.
- Refactor a graph by specializing relationships.
- Perform batch processing to refactor a large graph.

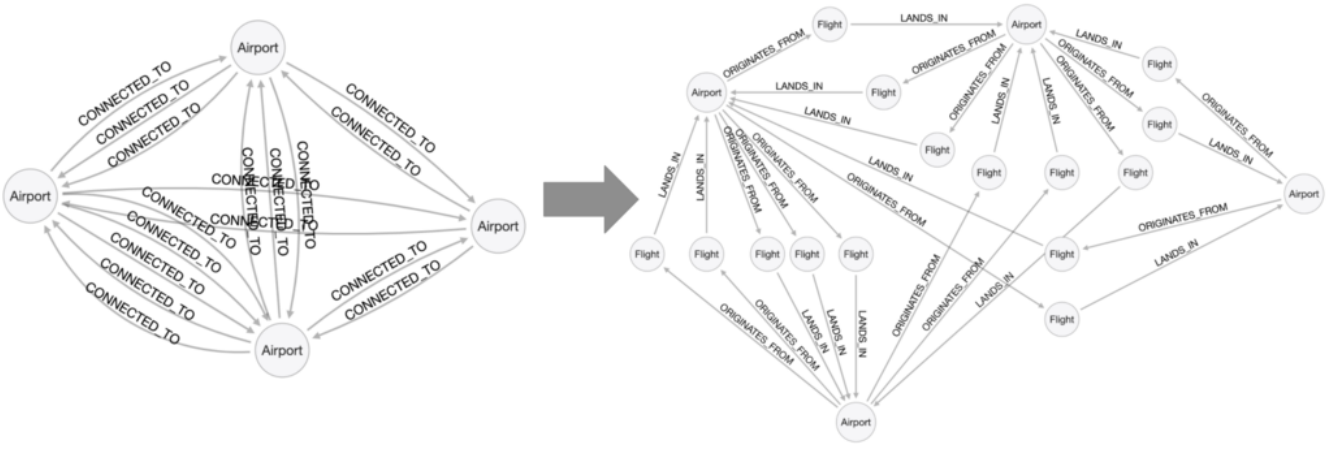
Evolving the model

After profiling queries and determining that they are not performing optimally, you should rethink your graph data model. You do so by examining how queries perform, especially with data at scale.

In the previous lesson, we saw that the query to retrieve the airports and flight information for flight number 1016 for airline WN ended up making two passes through the graph to perform the query. Our small database has only 100 flights in it, but it required over 4,000 db hits to perform the query.

To solve this problem, we introduce the intermediate node, Flight that is based upon the properties of the *CONNECTED_TO* relationship.

Here is how the model will evolve:



Refactoring steps

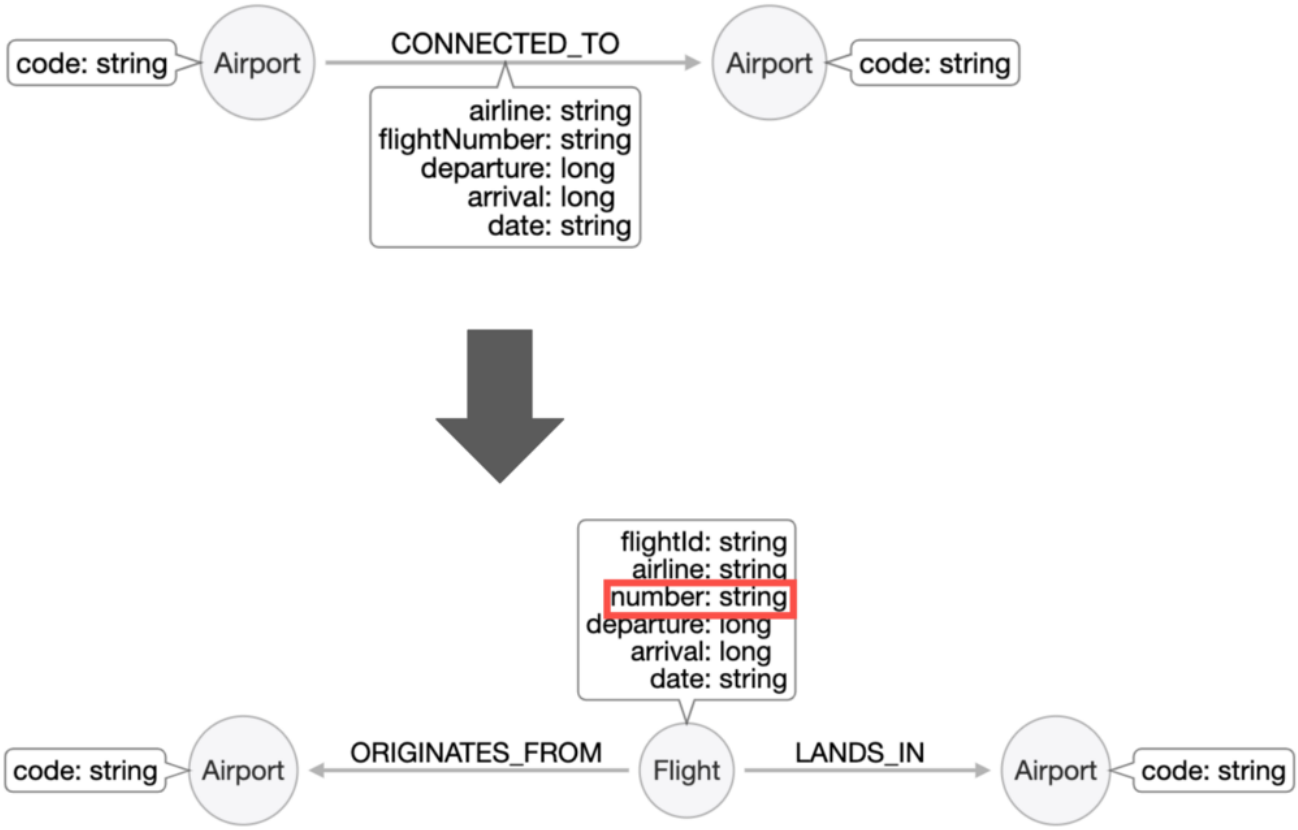
In most cases, you refactor the the implementation of the graph data model with these steps:

1. Create constraints as needed.
2. Execute the refactor:
    a. MATCH the data you wish to move.
    b. Create new nodes.
    c. Create new relationships.
3. Create indexes as needed.
4. PROFILE all queries against the new model.

NOTE	If the new model performs well for all queries, delete the old model. Otherwise, leave both in place.

Refactor details

Here are the details that show the *CONNECTED_TO* relationship properties and how they will be refactored as the Flight properties.



Notice that we will be adding a unique identifier for a *Flight* node, *flightId*.

Unique FlightId

Using a unique identifier for a flight enables us to quickly find a flight by its ID during a query. We create a uniqueness constraint for that property.

The flight number for a particular airline may not be unique, especially if we take into account the origin and destination airports. That is, the same flight number may be used for different from and

to airports. In addition, we want to uniquely identify the date for the flight.

The *Flight.flightId* property will contain the following data which will be unique for all *Flight* nodes:

- airline
- flightNumber
- code for the origin Airport
- code for the destination Airport
- date

Creating the constraint

Before we perform the refactoring of the existing graph, we add the uniqueness constraint:

```
CREATE CONSTRAINT Flight_flightId_constraint ON (f:Flight)
       ASSERT f.flightId IS UNIQUE
```

Refactoring the graph

Here is the code to refactor the graph:

```
MATCH (origin:Airport)-[connection:CONNECTED_TO]->(destination:Airport)
MERGE (newFlight:Flight {flightId: connection.airline +
connection.flightNumber +
       '_' + connection.date +  '_' + origin.code + '_' + destination.code
})
ON CREATE SET newFlight.date = connection.date,
              newFlight.airline = connection.airline,
              newFlight.number = connection.flightNumber,
              newFlight.departure = connection.departure,
              newFlight.arrival = connection.arrival
MERGE (origin)<-[:ORIGINATES_FROM]-(newFlight)
MERGE (newFlight)-[:LANDS_IN]->(destination)
```

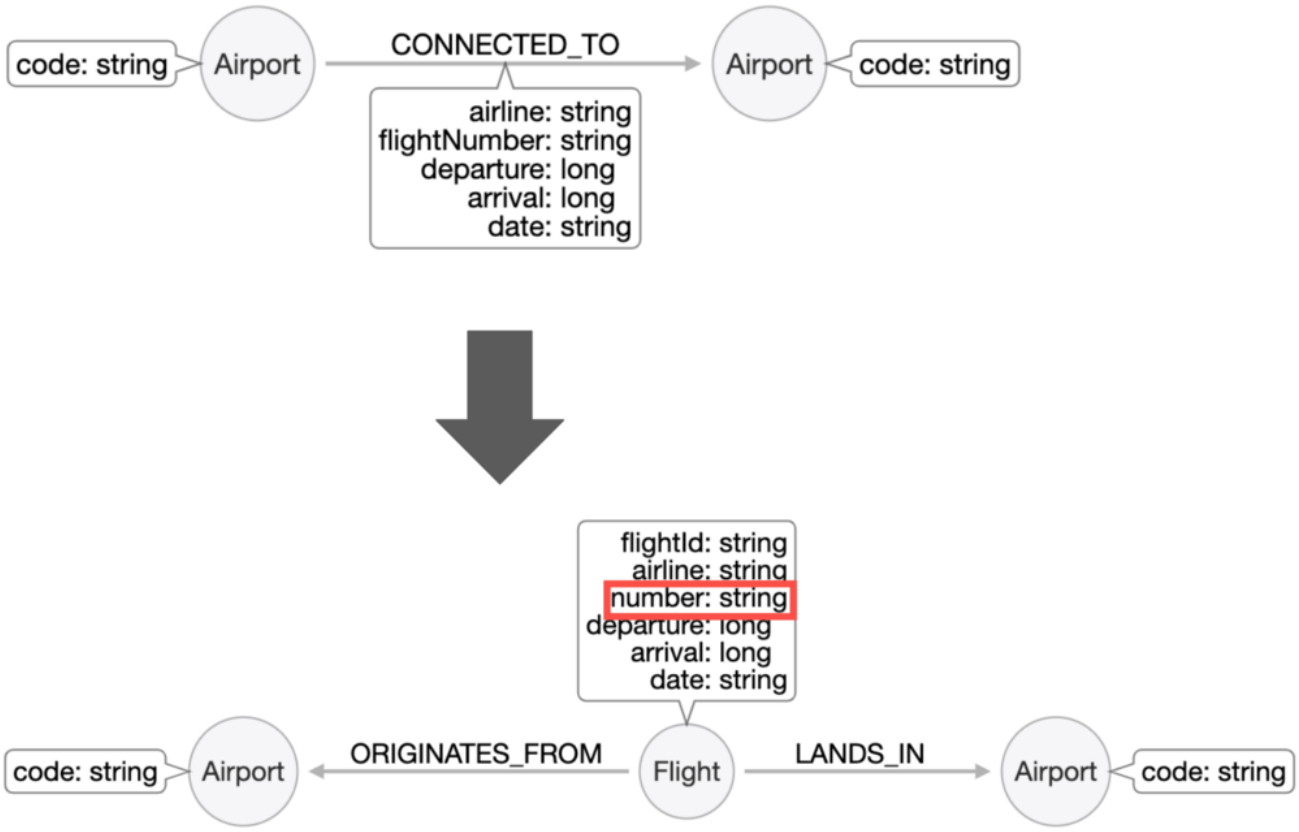
This code looks scary at first, but it relies entirely on Cypher code you already know well.

It follows our refactoring steps:

1. Use `MATCH` to fetch the data we wish to move. We are looking to move the *CONNECTED_TO* relationship data, so we create a query that will fetch every such relationship.
2. Use `MERGE` to create the new *Flight* nodes. As is always the best practice with `MERGE`, we only include the unique identifier, *flightId* in the `MERGE` statement, where that property is a concatenation of many individually-non-unique things. All other *Flight* properties are set in a subsequent `ON CREATE SET` clause, which maps 1:1 from the CONNECTED_TO properties.
3. Use `MERGE` to create the relationships connecting the new nodes. We need to do this twice, since Flights have both an ORIGINATES_FROM and LANDS_IN relationship.

Index for query performance

A very common qualifier for a query is the flight number. We want lookups of a *Flight* by its number to be fast. A flight number is not unique, but we want to be able to look up all flights with that number quickly.



You create the indexes after your have executed the refactoring code. This is because index maintenance is expensive and you do not want a large refactoring to take a long time to execute.

Creating the index

Here is the code to create the index on the flight number property:

```
CREATE INDEX Flight_number_index FOR (f:Flight) ON (f.number)
```

Profiling the query

After you have refactored the graph, you then profile the query that you are hoping to improve.

Recall that the query is: What are the airports and flight information for flight number 1016 for airline WN?

Here is the original query:

```
PROFILE
MATCH  (origin:Airport)-[connection:CONNECTED_TO]->(destination:Airport)
WHERE connection.airline = 'WN' AND connection.flightNumber = '1016'
RETURN origin.code, destination.code, connection.date,
connection.departure, connection.arrival
```

We must change the query to work with the new model as follows:

```
PROFILE
MATCH (origin)<-[:ORIGINATES_FROM]-(flight:Flight)-
      [:LANDS_IN]->(destination)
WHERE flight.airline = 'WN' AND
      flight.number = '1016' RETURN origin, destination, flight
```

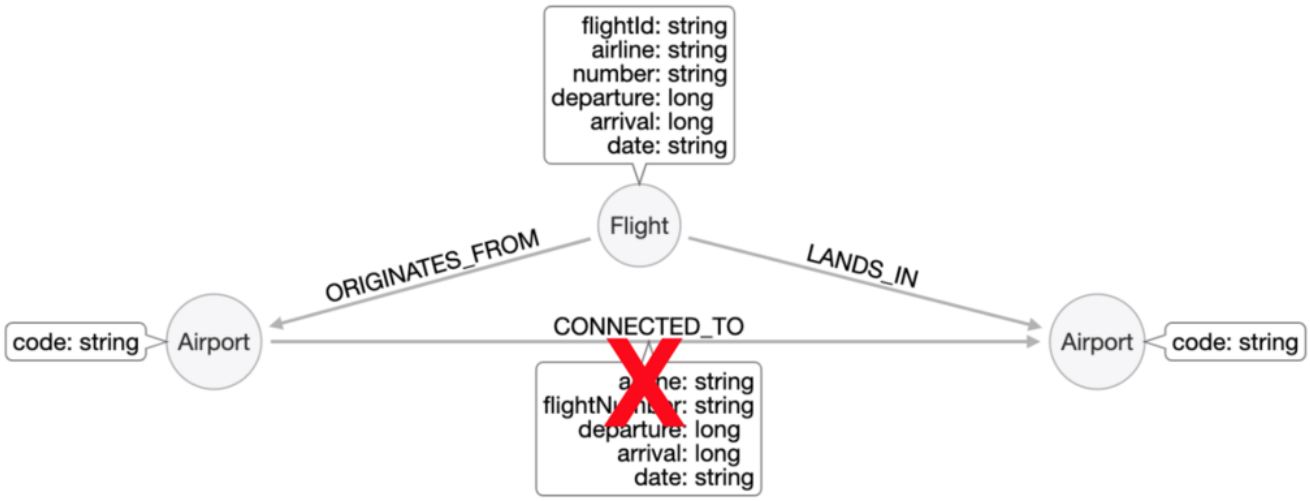
Result of the profile


NodeIndexSeek
flight
:Flight(number)
Ordered by flight.number ASC
1 estimated rows
4 db hits
3 rows
Filter
flight
flight.airline = $`AUTOSTRING0`
Ordered by flight.number ASC
0 estimated rows
12 db hits
3 rows
Expand(All)
destination, flight
(flight)-[UNNAMED51:LANDS_IN]→(destination)
Ordered by flight.number ASC
0 estimated rows
9 db hits
3 rows
Expand(All)
flight, origin, destination
(flight)-[UNNAMED15:ORIGINATES_FROM]→(origin)
Ordered by flight.number ASC
0 estimated rows
9 db hits
3 rows
ProduceResults
flight, origin, destination
Ordered by flight.number ASC
0 estimated rows
0 db hits
3 rows
Result
NED. 34 total db hits in 106 ms.

Here we see that we improved the query to require only 34 db hits. Notice also that the index is being used at the beginning of the query execution plan to anchor the query. It is much better than doing a NodeByLabelScan.

Cleaning up the graph

If you are satisfied that the new model performs best for your queries, you can then clean up the graph to remove the *CONNECTED_TO* relationships. Relationships, especially those with properties take up unnecessary space in the graph.




```
MATCH ()-[connection:CONNECTED_TO]->()
DELETE connection
```

Exercise 4: Creating Flight nodes from CONNECTED_TO relationships

In the query edit pane of Neo4j Browser, execute the browser command:

`:play 4.0-neo4j-modeling-exercises`

and follow the instructions for Exercise 4.

NOTE	This exercise has 7 steps. Estimated time to complete: 30 minutes.

Adding another domain question

We need to add another question for our application:

As a frequent traveller I want to find flights from <origin> to <destination> on <date> so that I can book my business flight.

For example:

Find all the flights going from Los Angeles (LAS) to Chicago Midway International (MDW) on the

3rd January, 2019.

Implementing the query

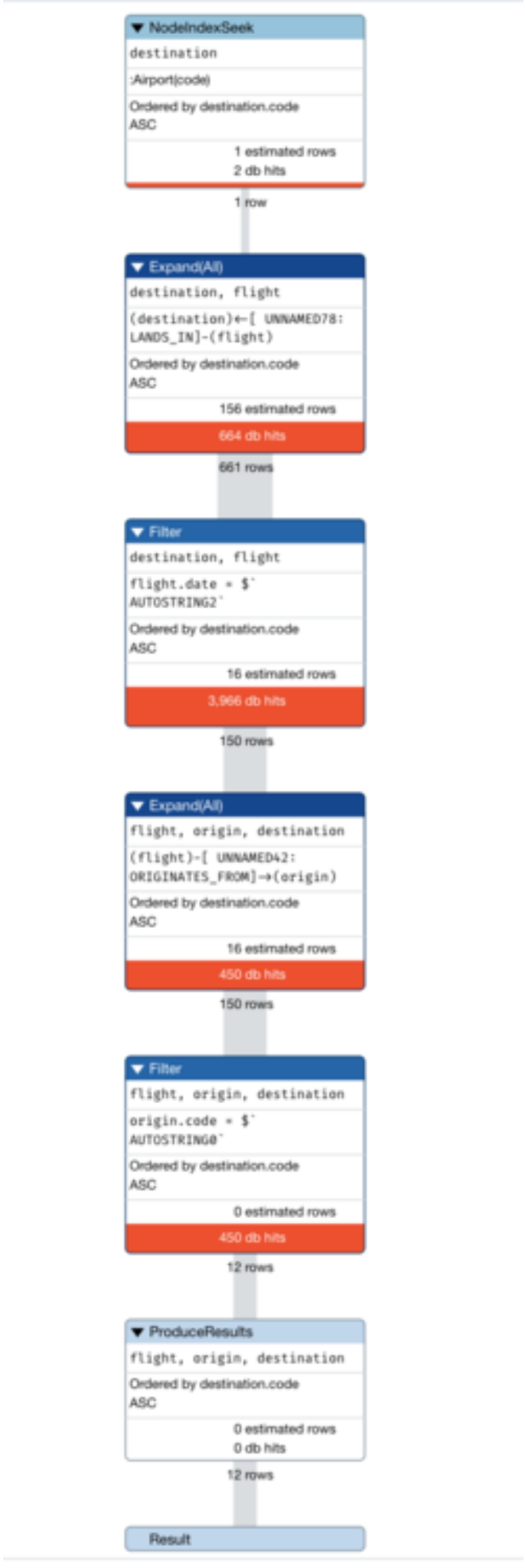
Here is the query:

```
MATCH (origin:Airport {code: 'LAS'})
    <-[:ORIGINATES_FROM]-(flight:Flight)-[:LANDS_IN]->
    (destination:Airport {code: 'MDW'})
WHERE flight.date = '2019-1-3'
RETURN origin, destination, flight
```

NOTE	You will work with this query in the next exercise where you have loaded 10k nodes into the graph.

Profiling the query

Here is the result of the profile on a graph that contains 10k nodes.


NodeIndexSeek
destination
:Airport(code)
Ordered by destination.code ASC
1 estimated rows
2 db hits
1 row
Expand(All)
destination, flight
(destination)<-[UNNAMED78: LANDS_IN]-(flight)
Ordered by destination.code ASC
156 estimated rows
664 db hits
661 rows
Filter
destination, flight
flight.date = $` AUTOSTRING2`
Ordered by destination.code ASC
16 estimated rows
3,966 db hits
150 rows
Expand(All)
flight, origin, destination
(flight)-[UNNAMED42: ORIGINATES_FROM]->(origin)
Ordered by destination.code ASC
16 estimated rows
450 db hits
150 rows
Filter
flight, origin, destination
origin.code = $` AUTOSTRING0`
Ordered by destination.code ASC
0 estimated rows
450 db hits
12 rows
ProduceResults
flight, origin, destination
Ordered by destination.code ASC
0 estimated rows
0 db hits
12 rows
Result
5532 total db hits in 33 ms.

This query returns 12 flights.

5532 db hits which needs improvement. Adding an index to *Flight* on date does not improve the query.

Some stats about this graph:

- 64 airports
- 10000 flights
- 661 flights that land in MDW; 377 on 2019-1-3
- 1624 flights that originate from LAS; 426 on 2019-1-3
- 2367 flights on 2019-1-3

Searching for flights by day is a problem.

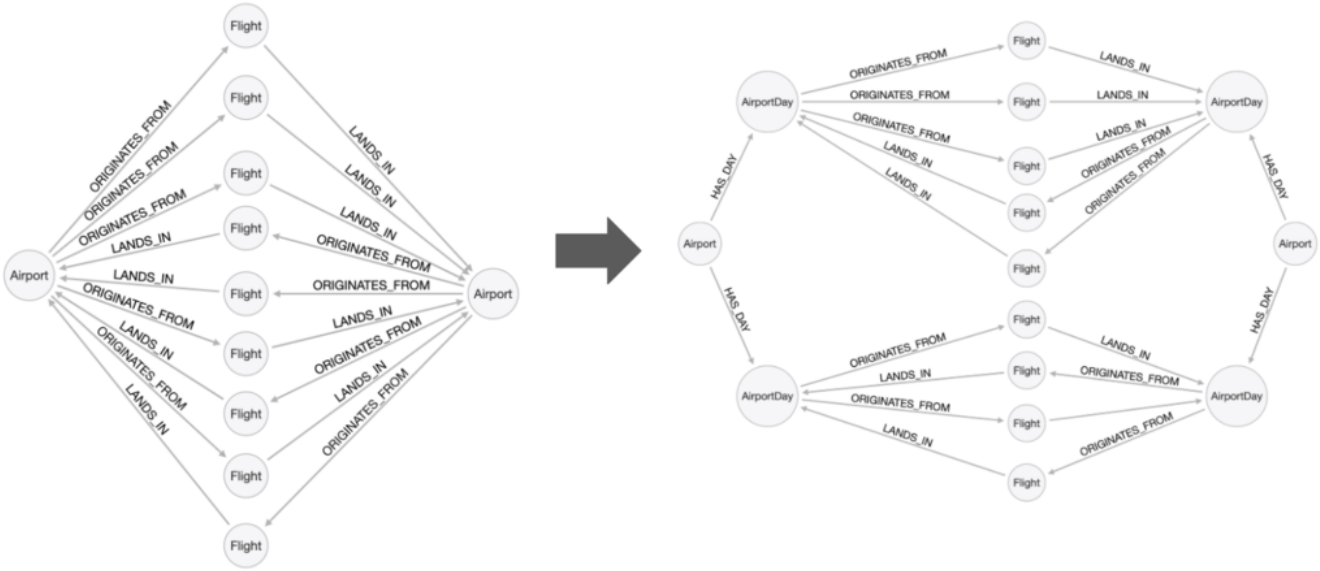
The query starts by using an index to find MDW but then has to traverse all incoming *LANDS_IN* relationships and check the *Flight.date* property on the other side. The more flights an airport has the more we will have to scan through, and since we are only working with 10,000 flights we should probably find a better way to model our data before importing any more data.

This is an opportunity to change the model.

Performing another refactor

We want to introduce *AirportDay* nodes so that we do not have to scan through all the flights going from an airport when we are only interested in a subset of them.

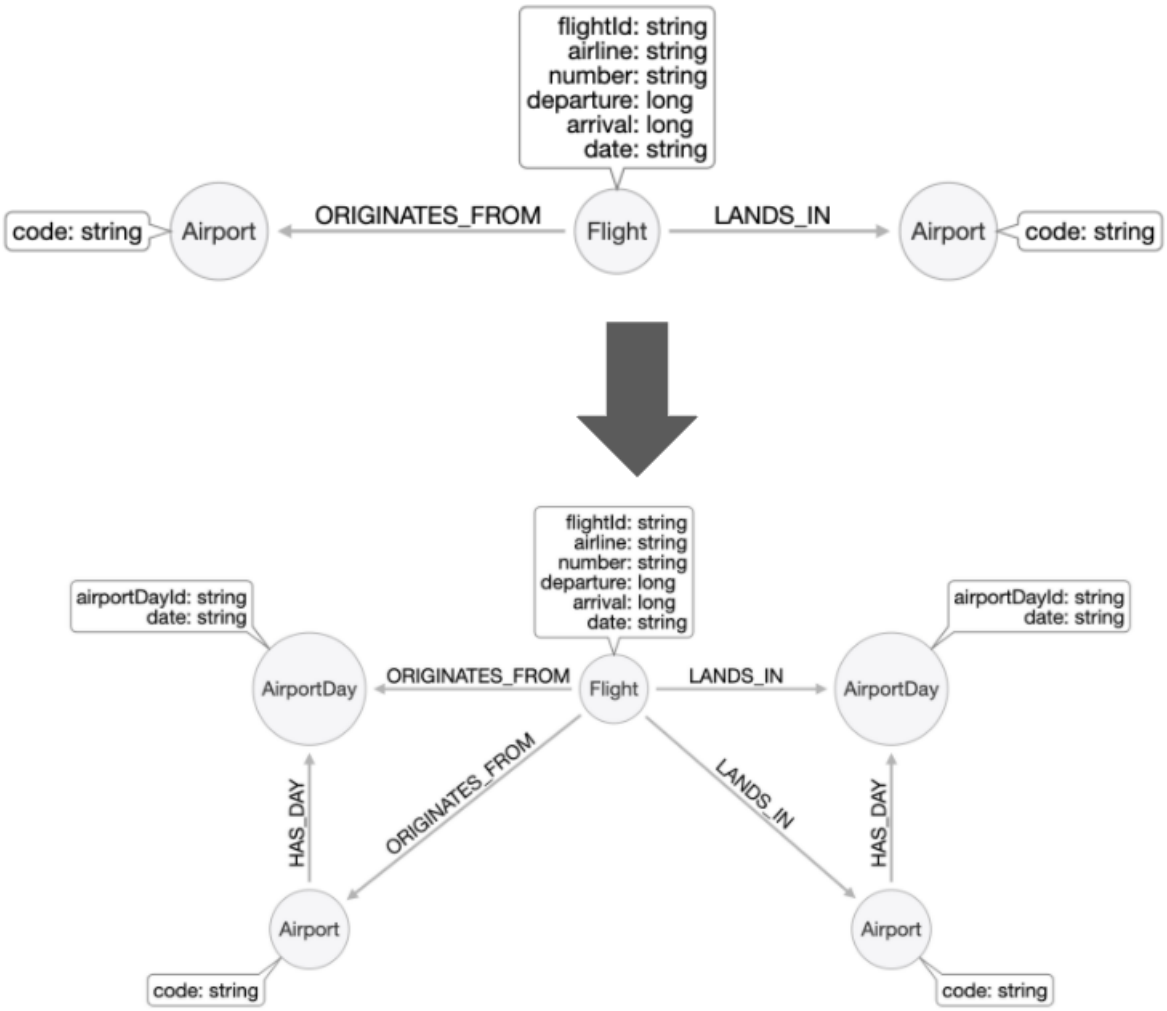
This is an instance where we do not want to remove the relationships between airports and flights because we need them for our first query "What are the airports and flight information for flight number 1016 for airline WN?".



NOTE	You only pull out a node if you are going to query through it, otherwise a property will suffice.

You should not be too aggressive with creating nodes from other nodes. If you pull out every single property and create nodes then you end up with an RDF model and lose the benefit of the property graph.

Refactor details



In this case we are adding the *AirportDay* node that will have date information. That way we do not have to go through the *Flight* nodes to find a date. Just like the *Flight* nodes, it will have a unique ID, *AirportDay.airportDayId* so that it can used it in the query.

```
CREATE CONSTRAINT AirportDay_airportDayId_constraint ON (a:AirportDay)
       ASSERT a.airportDayId IS UNIQUE
```

Then, you will want to:

- `MATCH` the data you want to move.
- Create the new *AirportDay* nodes.
- Connect the new nodes to the existing graph.

Refactor implementation

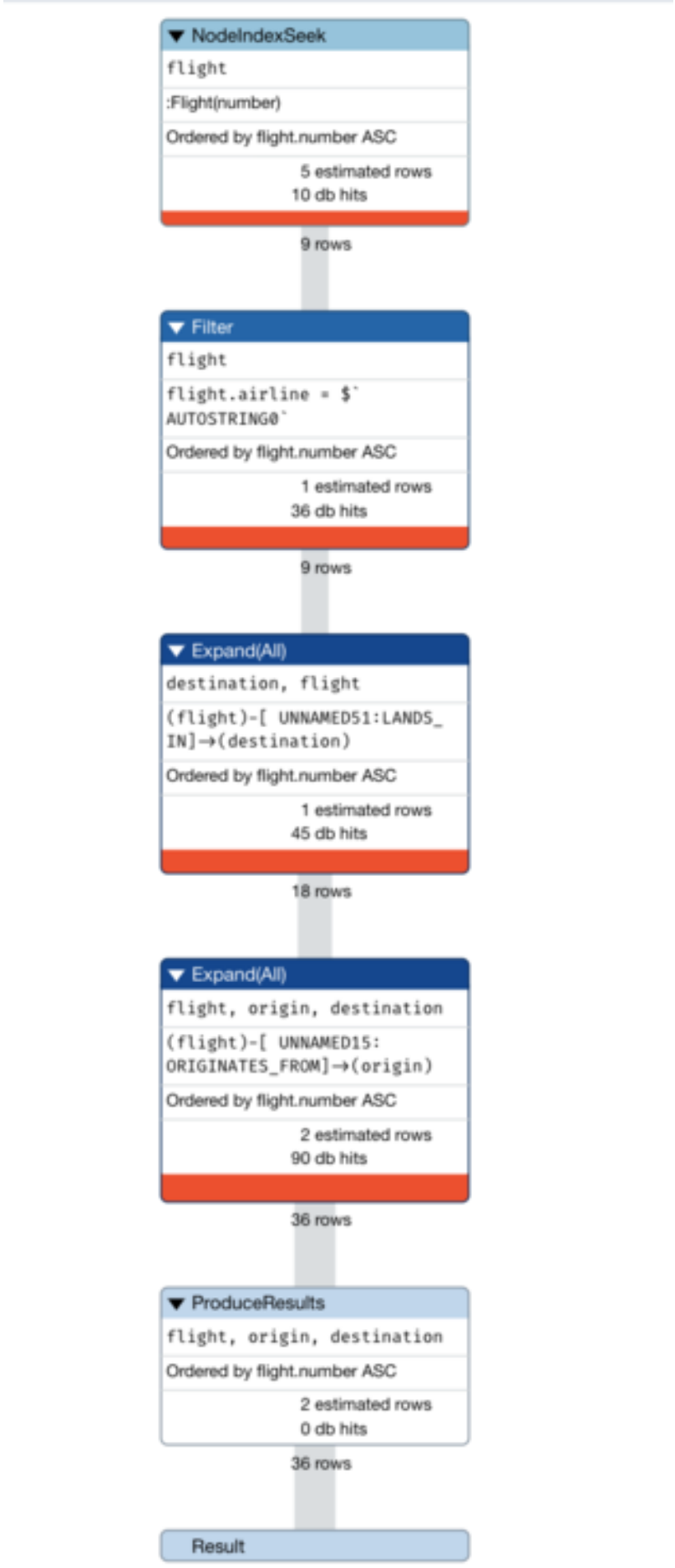
```
MATCH (origin:Airport)<-[:ORIGINATES_FROM]-(flight:Flight)-
      [:LANDS_IN]->(destination:Airport)
MERGE (originAirportDay:AirportDay {airportDayId: origin.code + '_' +
flight.date})
SET originAirportDay.date = flight.date
MERGE (destinationAirportDay:AirportDay
      {airportDayId: destination.code + '_' + flight.date})
SET destinationAirportDay.date = flight.date
MERGE (origin)-[:HAS_DAY]->(originAirportDay)
MERGE (flight)-[:ORIGINATES_FROM]->(originAirportDay)
MERGE (flight)-[:LANDS_IN]->(destinationAirportDay)
MERGE (destination)-[:HAS_DAY]->(destinationAirportDay)
```

`MERGE` enables us to add a single *AirportDay* node per the *airportDayId* value and also ensure that only one relationship is created between a *Flight* and an *AirportDay* node.

Profile our first query

After a refactor, you should check that all queries perform OK. Here is our first query: What are the airports and flight information for flight number 1016 for airline WN?

```
PROFILE
MATCH (origin)<-[:ORIGINATES_FROM]-(flight:Flight)-
      [:LANDS_IN]->(destination)
WHERE flight.airline = 'WN' AND
      flight.number = '1016' RETURN origin, destination, flight
```



. 181 total db hits in 23 ms.

This query previously had 34 db hits, but now has 181. But, we added another 10K nodes to the graph so this is a reasonable outcome.

Profile our original second query

Then we want to profile our second query after the refactor: Find all the flights going from Los Angeles (LAS) to Chicago Midway International (MDW) on the 3rd January, 2019.

```
PROFILE MATCH (origin:Airport {code: 'LAS'})
    <-[:ORIGINATES_FROM]-(flight:Flight)-
    [:LANDS_IN]->
    (destination:Airport {code: 'MDW'})
WHERE flight.date = '2019-1-3'
RETURN origin, destination, flight
```


NodeIndexSeek
destination
:Airport(code)
Ordered by destination.code
ASC
1 estimated rows
2 db hits
1 row
Expand(All)
destination, flight
(destination)←[UNNAMED80:
LANDS_IN]-(flight)
Ordered by destination.code
ASC
156 estimated rows
664 db hits
661 rows
Filter
destination, flight
flight.date = $`
AUTOSTRING2`
Ordered by destination.code
ASC
16 estimated rows
3,966 db hits
150 rows
Expand(All)
flight, origin, destination
(flight)-[UNNAMED44:
ORIGINATES_FROM]→(origin)
Ordered by destination.code
ASC
31 estimated rows
750 db hits
300 rows
Filter
flight, origin, destination
origin.code = $`
AUTOSTRING0`
Ordered by destination.code
ASC
0 estimated rows
600 db hits
12 rows
ıtime: PIPELINED. 5982 total db hits in 34 ms.

Here we see that the query has 5982 db hits, which is worse than the 5532 we had earlier before the refactor. This is because we need to incorporate the new model into the query.

Profile our revised second query

Here is a rewrite of the second query: Find all the flights going from Los Angeles (LAS) to Chicago Midway International (MDW) on the 3rd January, 2019. Due to the change in the model, we must rewrite the query as:

```
PROFILE MATCH (origin:Airport {code: 'LAS'})-
    [:HAS_DAY]->(:AirportDay {date: '2019-1-3'})<-
    [:ORIGINATES_FROM]-(flight:Flight),
    (flight)-[:LANDS_IN]->
    (:AirportDay {date: '2019-1-3'})<-
    [:HAS_DAY]-(destination:Airport {code: 'MDW'})
RETURN origin, destination, flight
```


NodeIndexSeek
destination
:Airport(code)
Ordered by destination.code ASC
1 estimated rows
2 db hits
1 row
Expand(All)
destination
(destination)-[UNNAMED195:HAS_DAY]→(UNNAMED156)
Ordered by destination.code ASC
4 estimated rows
8 db hits
4 rows
Filter
destination
`anon[156]`.date = $`AUTOSTRING2`
Ordered by destination.code ASC
0 estimated rows
12 db hits
1 row
Expand(All)
flight, destination
(UNNAMED156)←[UNNAMED142:LANDS_IN]-(flight)
Ordered by destination.code ASC
16 estimated rows
153 db hits
150 rows
Filter
flight, destination
flight:Flight
Ordered by destination.code ASC
16 estimated rows
150 db hits
150 rows
ıtime: PIPELINED. 4271 total db hits in 63 ms.

Here we see that the query has 4271 db hits, which is better than the 5532 we had earlier before the refactor. As the number of nodes and relationships grows in the graph, these performance differences will be significant.

What we have learned is that we have to change the model and the query.

Exercise 5: Creating the AirportDay node from the Airport and Flight nodes

In the query edit pane of Neo4j Browser, execute the browser command:

```
:play 4.0-neo4j-modeling-exercises
```

and follow the instructions for Exercise 5.

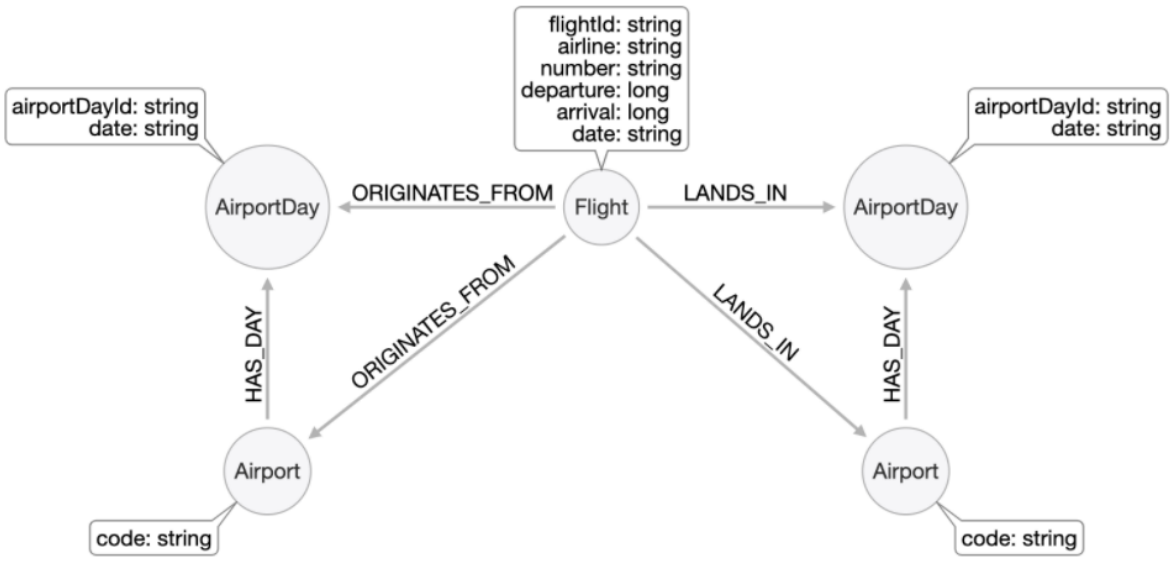
NOTE This exercise has 7 steps. Estimated time to complete: 30 minutes.

More questions for the model?

We now have a model that is performs well for these questions:

1. What are the airports and flight information for flight number xx for airline yy?
2. Find all the flights going from xx to yy on the date zz.

What if we added this question: Which airport has the most incoming flights?



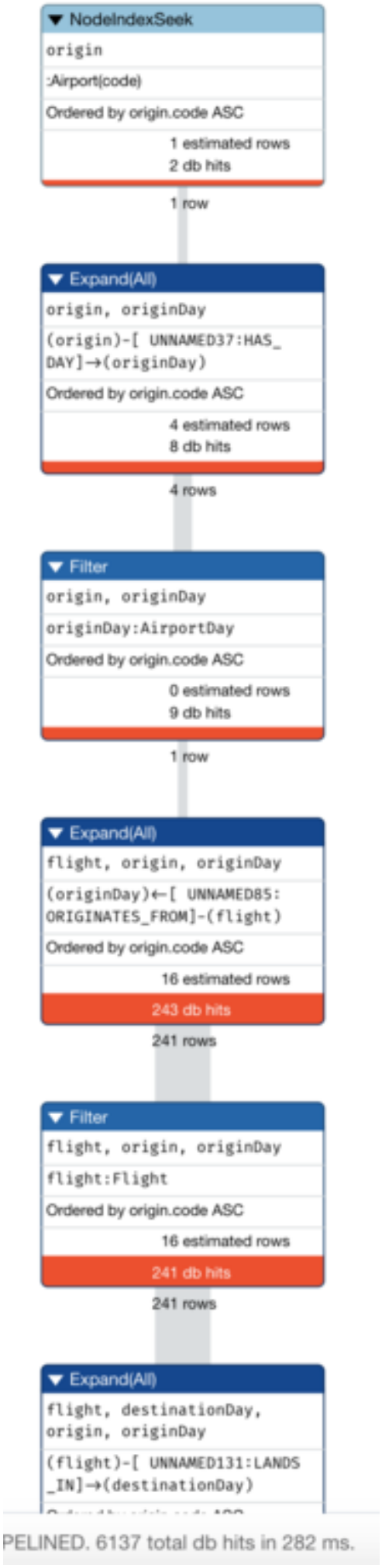
Our latest model serves our two questions very well. However, it does not do a good job with the new question. That is, we must leave the *ORIGINATES_FROM* and *LANDS_IN* relationships between *Airport* and *Flight* nodes in the graph. In this case, we leave both models in place, and use each one for the questions it is suited to.

Another question for the model

Suppose we added this question: What are the flights from LAS that arrive at MDW on 2019-1-3?

To answer this question with the current model our query would be:

```
PROFILE
MATCH (origin:Airport {code: 'LAS'})-[:HAS_DAY]->
      (originDay:AirportDay),
      (originDay)<-[:ORIGINATES_FROM]-(flight:Flight),
      (flight)-[:LANDS_IN]->(destinationDay),
      (destinationDay:AirportDay)<-[:HAS_DAY]-
      (destination:Airport {code: 'MDW'})
WHERE originDay.date = '2019-1-3' AND
destinationDay.date = '2019-1-3'
RETURN flight.date, flight.number, flight.airline,
flight.departure, flight.arrival
ORDER BY flight.date, flight.departure
```


NodeIndexSeek
origin
:Airport(code)
Ordered by origin.code ASC
1 estimated rows
2 db hits
1 row
Expand(All)
origin, originDay
(origin)-[UNNAMED37:HAS_
DAY]→(originDay)
Ordered by origin.code ASC
4 estimated rows
8 db hits
4 rows
Filter
origin, originDay
originDay:AirportDay
Ordered by origin.code ASC
0 estimated rows
9 db hits
1 row
Expand(All)
flight, origin, originDay
(originDay)←[UNNAMED85:
ORIGINATES_FROM]-(flight)
Ordered by origin.code ASC
16 estimated rows
243 db hits
241 rows
Filter
flight, origin, originDay
flight:Flight
Ordered by origin.code ASC
16 estimated rows
241 db hits
241 rows
Expand(All)
flight, destinationDay,
origin, originDay
(flight)-[UNNAMED131:LANDS
_IN]→(destinationDay)
PELINED. 6137 total db hits in 282 ms.

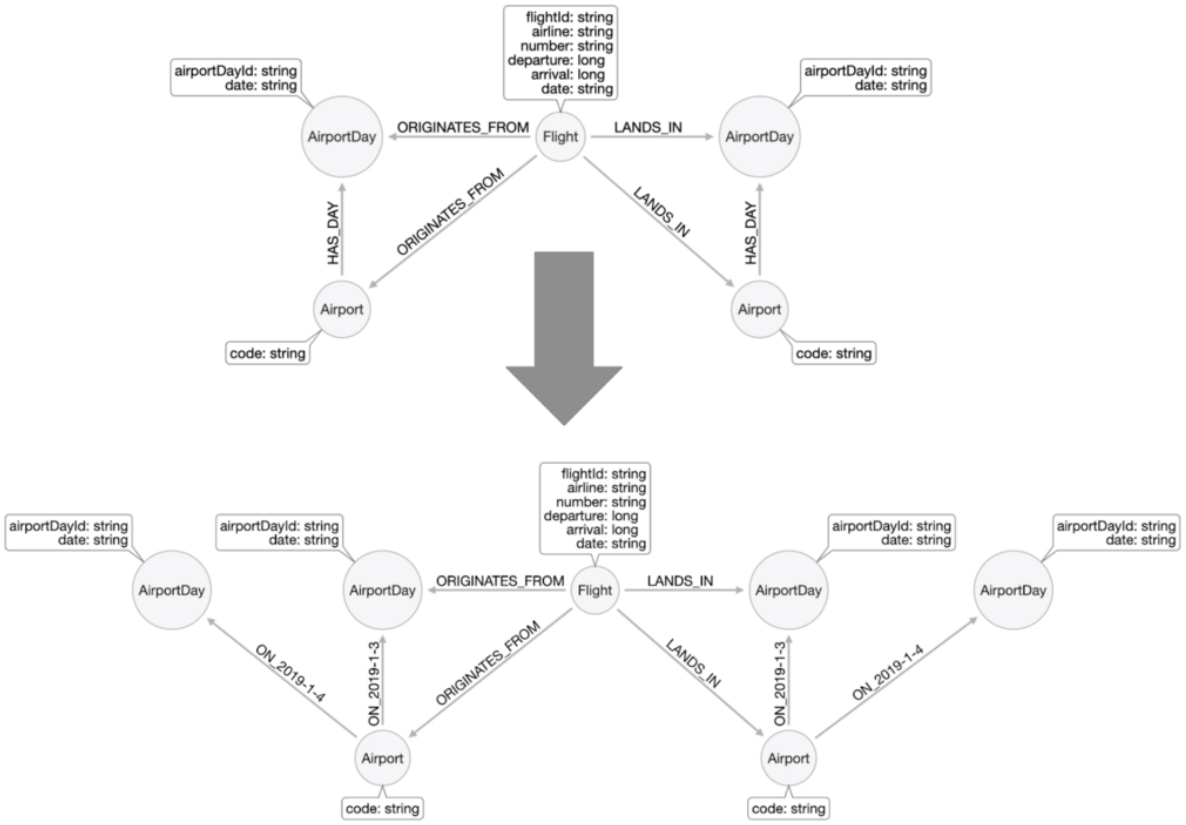
This query requires 6137 db hits with our current graph. It needs to traverse all the *HAS_DAY* relationships between the *Airport* and *AirportDay* nodes found.

Neo4j is optimized for searching by relationship types. As we add more data, the number of *HAS_DAY* relationships that we have to traverse increases.

If we have 10 years worth of data we have to traverse 3,650 relationships from the *Airport* to find the *AirportDay* that we are interested in.

Refactoring for specific relationships

A best practice for graph data modeling is to make relationships more specific if that will help with query performance. Here we can modify the *HAS_DAY* relationship to be *ON_2019-1-3*, *ON_2019_104*, etc.



APOC to the rescue!

With APOC, you can create relationships based upon data in the graph.

Here is the syntax:

```
apoc.create.relationship(startNode(<relationship-variable>),
                         '<new-relationship-value>',
                         {<relationship-property list},
                         endNode(<relationship-variable>)
                         )
                         YIELD rel
```

Given a relationship variable between two existing nodes, this procedure enables you to create a new, custom relationship that could be based upon property values. Calling this procedure where *rel* is returned enables you to either return the new relationship created or return a count of the number of relationships created in the graph.

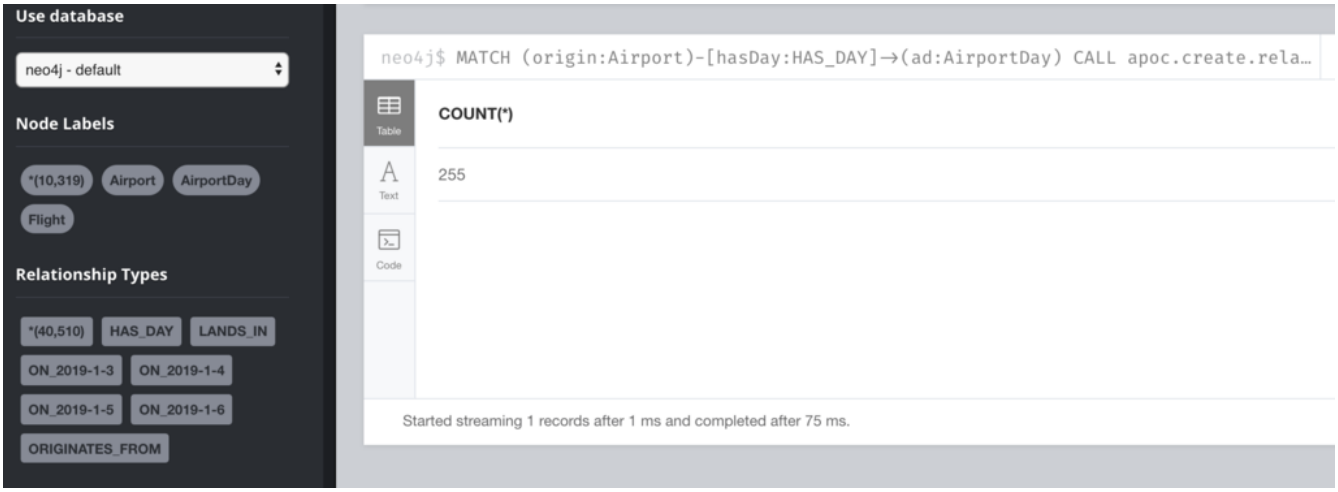
Creating specialized relationships with APOC

Here is the code to transform the *HAS_DAY* relationships to specific relationships:

```
MATCH (origin:Airport)-[hasDay:HAS_DAY]->(ad:AirportDay)
CALL apoc.create.relationship(startNode(hasDay),
                              'ON_' + ad.date,
                              {},
                              endNode(hasDay) ) YIELD rel
RETURN COUNT(*)
```

Refactoring result

Here is the result of the refactoring to create specific relationships:



Does the query improve?

Since the model has changed, we need to rewrite the query:

```
PROFILE
MATCH (origin:Airport {code: 'LAS'})-[:`ON_2019-1-3`]->
      (originDay:AirportDay),
      (originDay)<-[:ORIGINATES_FROM]-(flight:Flight),
      (flight)-[:LANDS_IN]->(destinationDay),
      (destinationDay:AirportDay)<-[:`ON_2019-1-3`]
      -(destination:Airport {code: 'MDW'})
RETURN flight.date, flight.number, flight.airline,
flight.departure, flight.arrival
ORDER BY flight.date, flight.departure
```


NodeIndexSeek
destination
:Airport(code)
Ordered by destination.code
ASC
1 estimated rows
2 db hits
1 row
Expand(All)
destination, destinationDay
(destination)-[UNNAMED196:
ON_2019-1-3]→(
destinationDay)
Ordered by destination.code
ASC
1 estimated rows
6 db hits
1 row
Filter
destination, destinationDay
destinationDay:AirportDay
Ordered by destination.code
ASC
1 estimated rows
1 db hit
1 row
Expand(All)
flight, destinationDay,
destination
(destinationDay)←[
UNNAMED137:LANDS_IN]-(flight
)
Ordered by destination.code
ASC
39 estimated rows
153 db hits
150 rows
PIPELINED. 4108 total db hits in 107

This query required 4108 db hits, where previously, it required 6137 hits. You can imagine that a fully-loaded graph with years of data could be vastly improved with this type of refactoring.

Of course, you must do due diligence and ensure that all of the previous queries still perform well.

Exercise 6: Creating specific relationships

In the query edit pane of Neo4j Browser, execute the browser command:

`:play 4.0-neo4j-modeling-exercises`

and follow the instructions for Exercise 6.

NOTE	This exercise has 2 steps. Estimated time to complete: 15 minutes.

Refactoring large graphs

Suppose you scale the test data in your graph, or you have a large production graph and a new question must be added that requires a change to the model. Refactoring a large graph has its challenges due to the amount of memory required to perform the refactor.

Cypher keeps all transaction state in memory while running a query, which is fine most of the time.

When refactoring the graph, however, this state can get very large and may result in an *OutOfMemory* exception.

You must adapt your heap size to match, or operate in batches. For example increase these values for the server in the **neo4j.conf** file:

- dbms.memory.heap.initial_size=2G (default is 512m)
- dbms.memory.heap.max_size=2G (default is 1G)

Batching the refactoring process

Here is one way that you can control how much work is done for a refactoring:

1. Tag all the nodes we need to process with a temporary label (for example *Process*).

```
MATCH (f:Flight)
SET f:Process
```

2. Iterate over a subset of nodes flagged with the temporary label (using `LIMIT`):
    a. Execute the refactoring code.
    b. Remove the temporary label from the nodes.
    c. Return a count of how many rows were processed in the iteration.
3. Once the count reaches 0, then the refactoring is finished.

Example code for a batch

Here is code that will process all nodes with the label *Process*:

```
MATCH (flight:Process)
WITH flight LIMIT 500

MATCH (origin:Airport)<-[:ORIGINATES_FROM]-(flight)-[:LANDS_IN]->
(destination:Airport)

MERGE (originAirportDay:AirportDay {airportDayId: origin.code + "_" +
flight.date})
ON CREATE SET originAirportDay.date = flight.date

MERGE (destinationAirportDay:AirportDay {airportDayId: destination.code +
"_" + flight.date})
ON CREATE SET destinationAirportDay.date = flight.date

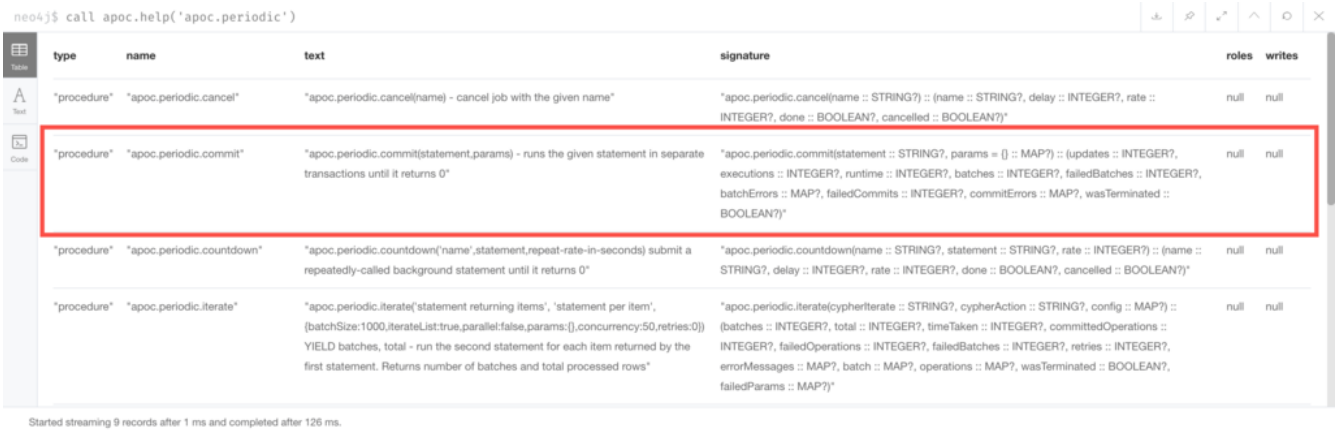
MERGE (origin)-[:HAS_DAY]->(originAirportDay)
MERGE (originAirportDay)<-[:ORIGINATES_FROM]-(flight)
MERGE (flight)-[:LANDS_IN]-(destinationAirportDay)
MERGE (destination)-[:HAS_DAY]->(destinationAirportDay)

REMOVE flight:Process
RETURN count(*)
```

You have previously used this refactoring code to create the *AirportDay* nodes and their relationships to *Airport* and *Flight* nodes.

This is a variation of the code we executed previously to create the *AirportDay* nodes from *Flight* nodes. The highlighted areas will be part of the iteration where we do batches of 500 flights at a time and once the creation of the *AirportDay* node is completed, we remove the *Process* label from the *Flight* node.

Using apoc.periodoc.commit



APOC currently has 9 procedures that can help you with batching. We will focus on using *apoc.periodic.commit()*.

Batching with APOC

```
CALL apoc.periodic.commit('
MATCH (flight:Process)
WITH flight LIMIT $limit

MATCH (origin:Airport)<-[:ORIGINATES_FROM]-(flight)-[:LANDS_IN]-
>(destination:Airport)

MERGE (originAirportDay:AirportDay {airportDayId: origin.code + "_" +
flight.date})
ON CREATE SET originAirportDay.date = flight.date

MERGE (destinationAirportDay:AirportDay {airportDayId: destination.code +
"_" + flight.date})
ON CREATE SET destinationAirportDay.date = flight.date

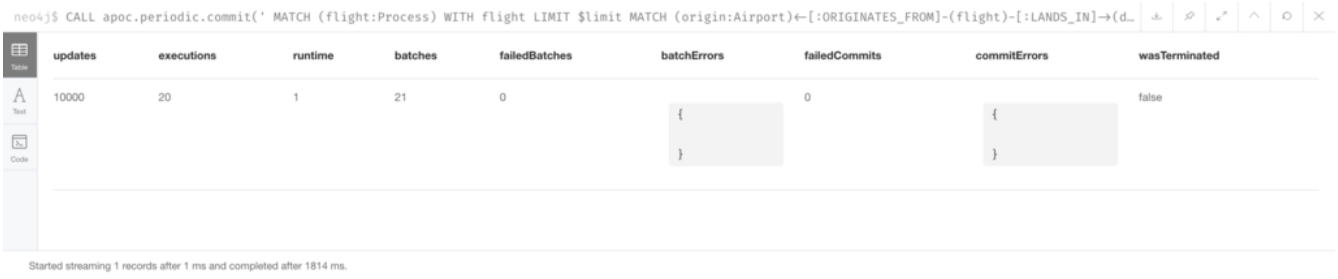
MERGE (origin)-[:HAS_DAY]->(originAirportDay)
MERGE (originAirportDay)<-[:ORIGINATES_FROM]-(flight)
MERGE (flight)-[:LANDS_IN]-(destinationAirportDay)
MERGE (destination)-[:HAS_DAY]->(destinationAirportDay)

REMOVE flight:Process
RETURN count(*)

',{limit:500}
)
```

Here we include the refactoring code for creating the *AirportDay* nodes/relationships in our call to *apoc.periodic.commit()* where we specify that the refactoring code will create 500 *AirportDay* nodes in a single transaction.

Result of the batch processing



Exercise 7: Refactoring large graphs

In the query edit pane of Neo4j Browser, execute the browser command:

:play 4.0-neo4j-modeling-exercises

and follow the instructions for Exercise 7.

NOTE	This exercise has 8 steps. Estimated time to complete: 30 minutes.

Check your understanding

Question 1

Suppose you want to create *Person* nodes in the graph, each with a unique value for the *personID* property . What must you do to ensure that nodes are unique?

Select the correct answers.

- ☐ Test the existence of the *Person* node with the *personID* property value before you use `CREATE` to create it.
- ☐ Create an existence constraint for the *personID* property of the *Person* node.
- ☑ Use `MERGE` to create the *Person* node with a unique property value specified for *personID*.
- ☑ Create a uniqueness constraint for the *personID* property of the *Person* node.

Question 2

In many cases, how will refactoring change the graph?

Select the correct answers.

- ☑ Add more nodes to the graph to represent the same data.
- ☐ Reduce nodes by consolidating data.
- ☑ Add more relationships to the graph.
- ☐ Reduce the number of relationships in the graph.

Question 3

What is the recommended method for batching refactorings for a large graph?

Select the correct answer.

- ☐ Use the `FOREACH` clause in Cypher.
- ☐ Use a `DO WHILE` clause in Cypher.
- ☐ Use *db.iterate()*.
- ☑ Use *apoc.periodic.commit()*.

Summary

You should now be able to:

- Create constraints that help with performance of node creation and queries.
- Determine if a query can be improved.
- Write Cypher code to refactor the data model in the graph.

- Create indexes that will help with query performance.
- Refactor a graph by creating intermediate nodes.
- Refactor a graph by specializing relationships.
- Perform batch processing to refactor a large graph.

Implementing Graph Data Models: Summary

Course Summary

In this course, you have learned how to:

- Implement a simple graph data model.
- Import data into an existing graph data model.
- Profile query performance against the implemented graph.
- Refactor graph data models

Resources

There are many resources available to you for learning more about Neo4j and Cypher.

https://neo4j.com/developer/resources/

Neo4j Community Site where you can ask or answer questions about Neo4j and discuss with other users:

https://community.neo4j.com

Neo4j documentation:

https://neo4j.com/docs/

Neo4j Sandboxes for experimenting with graphs:

https://sandbox.neo4j.com

Videos on the Neo4j YouTube channel:

https://www.youtube.com/channel/UCvze3hU6OZBkB1vkhH2lH9Q

Neo4j online and classroom training:

https://neo4j.com/graphacademy/

Become a Neo4j certified developer:

https://neo4j.com/graphacademy/neo4j-certification/

NOTE	The certification exam tests you on content from this course. You can take the certification exam multiple times until you pass!

GitHub repository:

https://github.com/neo4j-contrib

Neo4j events all over the world:

https://neo4j.com/events/world/all/

Graph Gists for learning more use cases for Neo4j:

https://neo4j.com/graphgists/

Attend a Neo4j meetup:

https://www.meetup.com/topics/neo4j/

View questions/answers raised about Neo4j:

https://stackoverflow.com/tags/neo4j/hot

Course feedback

We want your feedback on this course. Please provide your feedback so we can improve this course.